
Exploring Transfer Learning

Diogo Paulo

Royal Institute of Technology, Sweden
diogop@kth.se

Hugo Dezerto

Royal Institute of Technology, Sweden
hugoad@kth.se

Maria Carolina Sebastião

Royal Institute of Technology, Sweden
mcms2@kth.se

Abstract

1 This project investigates efficient transfer learning strategies for image classifica-
2 tion on the Oxford-IIIT Pet Dataset. We begin by establishing a strong ResNet18-
3 based baseline. To address the challenge of limited labelled data, we explore
4 semi-supervised learning using pseudo-labeling, demonstrating significant im-
5 provements when only a small fraction of data is labeled. We further evaluate
6 the performance of Vision Transformers (ViTs), which show superior accuracy
7 compared to convolutional baselines. Finally, we introduce Low-Rank Adaptation
8 (LoRA) layers into frozen ResNet backbones, achieving competitive results while
9 using less trainable parameters. Our experiments highlight the effectiveness and
10 efficiency of these techniques in our specific classification setting.

1 Introduction

Transfer learning is a machine learning technique based on the idea to use knowledge gathered from other tasks to solve a new one. This is especially useful when data for the target task is limited. More recently, Vision Transformers have extended the transfer learning paradigm by pre-training on massive image datasets and fine-tuning efficiently on specific tasks, making them a compelling alternative to convolutional networks for scenarios with limited labelled data. ^[1]

In this report, we aim to improve a ResNet-based baseline on the 37-breed Oxford-IIIT Pet Dataset ^[2] by (1) using semi-supervised pseudo-labeling to exploit abundant unlabelled images, ^[3] (2) fine-tuning a Hugging Face Vision Transformer model to adapt powerful self-attention representations to our pet classification task ^[1] and (3) adding Low-Rank Adaptation layers into the frozen backbone to learn compact, task-specific parameters without full model updates. ^[4]

Code Repository: The code for all experiments is available [here](#).

2 Related Work

Transfer learning has become a standard approach in computer vision, with pre-trained convolutional neural networks (CNNs) such as ResNet achieving strong results on a variety of tasks. ^[5] Fine-tuning strategies, including selective layer unfreezing and differential learning rates, have been shown to improve adaptation to new domains. Vision Transformers ^[6] have recently emerged as a powerful alternative to CNNs, using self-attention to capture global context. Semi-supervised learning methods, such as pseudo-labeling, ^[7] enable the use of unlabelled data to improve performance when labelled data is limited. LoRA offers an efficient way to adapt large pre-trained models by introducing low-rank trainable adapters. ^[4] Our work builds on these advances by systematically comparing fine-tuning strategies, evaluating ViT and LoRA, and assessing the impact of semi-supervised learning on the Oxford-IIIT Pet Dataset.

3 Data

We use the Oxford-IIIT Pet Dataset ^[2], which contains 7,349 images of cats and dogs spanning 37 breeds. Each image is annotated with both a breed label (multi-class) and a binary label (cat vs. dog). The provided training set was further split, with 20% used for validation and the remaining 80% for training. The official test set from the dataset was used for final evaluation. All images were resized so the shortest side was 224 pixels, maintaining aspect ratio, and normalized using ImageNet statistics to match the pre-trained model’s expected input. Additional data augmentation strategies specific to each method are described in Section 4.

4 Methods

This section presents the baseline model and the methods explored to improve it. Results for both the baseline and all extensions are reported and compared in Section 5.

4.1 Baseline

As mentioned, we used a pre-trained ResNet18 convolutional neural network as the backbone for all experiments. For both binary and multi-class classification, we replaced the final fully connected (fc) layer to match the number of output classes: 2 for binary (cat vs. dog) and 37 for multi-class (breed classification).

For **binary classification**, we froze all pre-trained weights and trained only the new fc layer using the Adam optimizer and cross-entropy loss, relying on the pre-trained network’s feature extraction to achieve fast convergence and high accuracy with minimal training.

For the **multi-class classification** task, we explored two fine-tuning strategies:

- **Strategy 1 (Simultaneous Fine-Tuning):** We progressively unfroze and fine-tuned the last l residual blocks (from layer4 to layer1), along with the fc layer. The value of l that yielded the highest validation accuracy was selected.

57 • **Strategy 2 (Gradual Unfreezing):** We started by fine-tuning only the f.c layer and the last
58 block, then gradually unfroze earlier layers in stages, retraining after each unfreezing.

59 For both strategies, we experimented with data augmentation (random crops, flips, small rotations),
60 L2 regularization, different learning rates for the classifier and backbone and learning rate schedulers,
61 and different batch normalization modes (freezing parameters or running statistics). Performance
62 was monitored on a validation set to adjust learning rates during training, to select the best model
63 checkpoint for final evaluation (Strategy 1) and to manually guide hyper-parameter tuning.

64 Finally, to assess robustness to **class imbalance**, we created an imbalanced training set by reducing cat
65 breed samples to 20% of their original size. We compared standard training, weighted cross-entropy
66 loss, and over-sampling of minority classes to evaluate their effect on minority class performance.

67 4.2 Semi-Supervised Learning with Pseudo-Labeling

68 To address the challenge of limited labelled data, we extended the baseline with a semi-supervised
69 pseudo-labeling approach, using the same parameters as in the baseline setup. For each labelled data
70 fraction—specifically, 1%, 10%, 50% and 100% of the training set—we first trained the baseline
71 model for 15 epochs using only the labelled subset. The best model from this stage was then used to
72 generate pseudo-labels for the remaining unlabelled samples by assigning each image the class with
73 the highest predicted probability. These pseudo-labelled samples were combined with the original
74 labelled set, and the model was retrained for another 15 epochs on this augmented dataset. The
75 procedure was then repeated for each labelled data fraction to systematically evaluate the effect of
76 increasing the amount of labelled data.

77 This process builds upon the structure of the unlabelled data distribution to improve generalization
78 when labelled data is scarce, as pseudo-labeling encourages the decision boundary to pass through low-
79 density regions. ^[7] Furthermore, this approach allows us to incorporate information from unlabelled
80 data and investigate whether it is possible to achieve comparable performance to the fully-supervised
81 setting, even when only a small percentage of the training data is labelled. The effectiveness of this
82 method is evaluated across different fractions of labelled data, as detailed in Section 5.2.

83 4.3 Vision Transformers

84 In this part of the project, we explored the use of ViT for the image classification task, using the
85 ViT-Base-Patch16-224 model, pre-trained on ImageNet-21k, as provided by the HuggingFace
86 Transformers library. ^[8] Unlike convolutional neural networks, ViT models divide each image into
87 fixed-size patches, linearly embed these patches and process them as a sequence using transformer
88 encoders. This patch-based self-attention mechanism allows ViTs to capture long-range dependencies
89 and global context more effectively than traditional CNNs. ^[1]

90 When comparing to fine-tuning a ViT with a Hugging Face’s Trainer, one key advantage of fine-tuning
91 a ResNet is its efficiency and ease of optimization. Thanks to convolutional inductive biases, ResNet
92 models typically converge faster and require fewer labelled samples than Vision Transformers. ^[9]
93 Their architectures are well-understood, work reliably with standard optimizers and hyper-parameters
94 and are less sensitive to data augmentation and preprocessing choices. ^[10]

95 However, ResNet’s reliance on pretrained features from ImageNet can be a disadvantage when the
96 target domain differs significantly, often requiring careful layer freezing/unfreezing and extensive
97 hyper-parameter tuning. This process can be challenging and may harm generalization. In contrast,
98 ViTs—while more computationally demanding and sensitive to training setup—can, when sufficiently
99 pre-trained, match or surpass ResNet accuracy and capture richer global context. ^[11]

100 In our project, we fine-tuned a ViT model by replacing the classification head and training on our
101 dataset with the same optimization settings as for ResNet. For data augmentation, we extended the
102 baseline implementation by adding color jitter, random affine and random erasing transformations
103 during training to further improve generalization. The model was trained for 30 epochs and the best
104 checkpoint was selected based on validation accuracy. The final performance was evaluated on the
105 test set. The results and comparison with convolutional baselines are discussed in Section 5.3.

4.4 LoRA layers

In this approach, we employ Low-Rank Adaptation to enable efficient fine-tuning of pre-trained ResNet models. LoRA modifies the original weight matrix W as $W' = W + \frac{\alpha}{r}BA$, where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ are the trainable low-rank matrices, r is the rank (controlling the capacity of the adaptation), and α is a scaling factor that determines the strength of the update. This allows the model to learn task-specific updates in a low-dimensional subspace, making adaptation both memory and compute-efficient.

We introduced LoRA adapters into selected layers—specifically, the second convolutional layers within residual layers 2, 3, and 4—while keeping the original pre-trained weights frozen. During training, only the LoRA adapters and the final classification layer are updated, significantly reducing the number of trainable parameters.

We further explored the impact of LoRA hyper-parameters (r and α) and applied the method to deeper ResNet variants to analyze the trade-off between model depth, parameter efficiency and performance. Results and comparisons are presented in Section 5.4.

5 Experiments

All experiments were conducted on a Google Cloud instance (n1-standard-4, 4 vCPUs, 15 GB RAM) equipped with an NVIDIA T4 GPU.

5.1 Baseline

Binary classification: After fine-tuning, a test accuracy of **99.02%** was achieved, demonstrating the effectiveness of transfer learning with strong pre-trained features (see A.1 for more details).

Multi-class classification: After fine-tuning and conducting ablation studies (based on validation accuracy), Strategy 1 achieved a higher test accuracy (**90.54%**) compared to Strategy 2 (90.02%). Thus, the **baseline** is defined as (see A.2 for more details): the last three residual blocks (layer2, layer3, layer4) and the final fc layer are fine-tuned using the Adam optimizer with learning rates of 1×10^{-3} for the fc layer and 1×10^{-5} for the backbone. Training is performed for 15 epochs with a batch size of 64, using RandomResizedCrop(224) and RandomHorizontalFlip for data augmentation, no regularization ($\lambda = 0$) and a ReduceLROnPlateau scheduler (factor 0.1, patience 1). Batch normalization layers use the default (unfrozen) setting.

Imbalanced classes: Reducing the number of cat breed samples to 20% dropped the test accuracy to 86.86%. Using a weighted loss and over-sampling improved it to 87.68%.

Overall, the baseline demonstrates strong performance with careful fine-tuning and highlights the impact of augmentation strategies, learning rate tuning and class imbalance handling.

5.2 Semi-Supervised Learning Results

We evaluated the effectiveness of semi-supervised learning through pseudo-labeling by measuring the test accuracy of the model across different fractions of labelled data. For each fraction, we report the test accuracy obtained using only the labelled data (supervised) and after incorporating pseudo-labeled data (semi-supervised). The results are summarized in Table 1.

Table 1: Test accuracy for different fractions of labelled data, comparing supervised and semi-supervised (pseudo-labeling) approaches.

Fraction of Labelled Data	Supervised Accuracy (%)	Semi-Supervised Accuracy (%)
1%	42.9	51.5
10%	77.8	81.4
50%	89.0	89.0
100%	90.5	—

The results show that pseudo-labeling provides a substantial improvement in accuracy when only a small fraction of the data is labelled, particularly at the 1% and 10% levels. As the amount of labelled data increases, the benefit of pseudo-labeling decreases and the performance approaches that of the fully-supervised baseline. When all data is labelled, pseudo-labeling is not applicable. These findings highlight the value of semi-supervised learning when having only a small fraction of labelled data.

5.3 Vision Transformer Results

The performance of the ViT model was evaluated on the Oxford-IIIT Pet dataset using the experimental setup described earlier. After fine-tuning the model for 30 epochs and selecting the best checkpoint based on validation accuracy, we evaluated its final performance on the test set.

The ViT-Base-Patch16-224 model achieved a test accuracy of **92.34%**, demonstrating the strong capability of transformer-based architectures in image classification tasks. Notably, these results are superior to those obtained with the baseline, highlighting the effectiveness of ViT models when combined with appropriate data augmentation and optimization strategies.

5.4 LoRA results

Hyper-parameter analysis:

When adding LoRA layers to the classification pre-trained model, it is important to determine what LoRA hyper-parameters lead to the best accuracy. To this end, a small search was conducted to understand whether keeping a small rank is better than a large one, and whether it is preferable to use a small or large alpha value. The search results are presented in Table 2.

Table 2: Validation and test accuracy for different LoRA configurations.

r	α	Validation Accuracy (%)	Test Accuracy (%)
4	4	92.80	88.09
4	8	92.93	89.26
8	8	91.85	88.80
8	16	93.21	88.93

The highest validation accuracy was obtained with $r = 8, \alpha = 16$. However, increasing the rank also led to greater overfitting, as indicated by the larger gap between validation and test accuracy (4.28%). Interestingly, the configuration $r = 4, \alpha = 8$ outperformed all others in terms of the test accuracy. Therefore, as it is suggested in LoRA’s original paper, increasing rank does not necessarily cover more meaningful space. Keeping LoRA layers small can be sufficient. [4] It can also be noted that when the scaling factor (α/r) was larger, accuracy improved. A higher scaling factor increases the influence of the LoRA adapters on the model’s output, allowing them to make more significant updates during fine-tuning. This can help the model adapt better to the new task, especially when the LoRA rank is kept small.

Comparison with baseline:

Considering the pair of hyper-parameters that led to the best results, we can analyse the efficiency of the LoRA layers in terms of the number of trainable parameters.

Following the results obtained for the baseline model, we froze all pre-trained weights from the ResNet18 backbone and added LoRA adapters to layers 2, 3, and 4 to efficiently adapt the pre-trained backbone. The fully connected layer was replaced to match the number of classes and kept trainable, with 18,981 parameters.

When using LoRA with $r = 4$, only 71,680 additional trainable parameters are introduced in layers 2, 3, and 4—just about 0.65% of the baseline (11,019,008 parameters). These results demonstrate that even with a very small number of additional parameters, LoRA can achieve high accuracy, thus confirming its efficiency.

Comparison across different depth architectures:

To try to further explore the effect of adding LoRA layers, we decided to initialize different ResNet models. In Table 3, we can see that, interestingly, the number of trainable parameters introduced for ResNet34 and ResNet50 is the same, which in turn, is precisely twice the amount introduced for ResNet18. This occurs because we are adding LoRA adapters only to the convolution2 layers within residual layers 2, 3, and 4 of each model. The convolution2 layers have the same number of channels, kernel size, stride, padding and dilation across all models. ResNet18 has fewer blocks in residual layers 2, 3 and 4 (2-2-2 blocks per layer) compared to ResNet34 and ResNet50 (4-6-3 blocks per layer), thus resulting in fewer LoRA parameters.

Table 3: Comparison of the number of parameters introduced by adding LoRA layers to deep models, with $r = 4$, $\alpha = 8$.

Model	LoRA Params	Val Acc (%)	Test Acc (%)	GPU Max (MB)	Time (min)
ResNet18	71,680	92.39	89.26	575.41	3.04
ResNet34	143,360	92.12	91.17	3589.42	3.51
ResNet50	143,360	94.84	91.77	3589.42	5.76

We can conclude that using a deeper architecture improves test accuracy but at the cost of higher training time, memory and a higher number of parameters. In any case, it can still be deduced that adding LoRA layers is very efficient.

6 Conclusion

In this project, we explored the concept of transfer learning, by carefully fine-tuning a ResNet18 backbone—selectively unfreezing its later residual blocks and tuning learning rates—we established a strong baseline that achieved over 90% accuracy on the 37-breed Oxford-IIIT Pet Dataset.

Building on that, our pseudo-labeling experiments demonstrated that even when only a tiny fraction of images carry labels, using confident model predictions on unlabelled data can really boost performance—recovering nearly 9 % of accuracy when only 1 percent of the data was initially labeled. Moving to transformer-based architectures, we found that a ViT-Base model, when properly augmented and trained, surpassed our convolutional baselines, showing the power of self-attention for capturing global image context. Finally, by introducing Low-Rank Adaptation layers into frozen ResNet backbones, we showed that it is possible to adapt large models with less than 1 percent of the trainable parameters of the baseline, retaining high accuracy while greatly reducing computational and memory overhead.

Looking ahead, a promising direction for future work could be adding LoRA layers to Vision Transformers. This avenue could lead to higher accuracy, while keeping a small number of trainable parameters.

References

- [1] Nate Raw. Fine-tune vit for image classification with transformers. <https://huggingface.co/blog/fine-tune-vit>, February 2022. Accessed: 2025-05-19.
- [2] Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman, and C. V. Jawahar. The oxford-iiit pet dataset. <https://www.robots.ox.ac.uk/~vgg/data/pets/>. Accessed: 2025-05-19.
- [3] F. Zhuang, Z. Qi, K. Duan, D. Xi, Y. Zhu, H. Zhu, H. Xiong, and Q. He. A comprehensive survey on transfer learning. *arXiv:1911.02685*, 2019. Accessed: 2025-05-19.
- [4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, and W. Chen. Lora: Low-rank adaptation of large language models. *arXiv:2106.09685*, 2021. Accessed: 2025-05-19.
- [5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *arXiv:1512.03385*, 2015.
- [6] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv:2010.11929*, 2020.
- [7] D.-H. Lee. Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks. Workshop on Challenges in Representation Learning, ICML, 2013.
- [8] Google Research. Vit-base patch16-224. <https://huggingface.co/google/vit-base-patch16-224>. Accessed: 2025-05-17.
- [9] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *Proceedings of NeurIPS*, 2021.
- [10] Tong He, Zhi Zhang, Hang Zhang, Zhongyue Zhang, Junyuan Xie, and Mu Li. Bag of tricks for image classification with convolutional neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 558–567, June 2019. Available: https://openaccess.thecvf.com/content_CVPR_2019/papers/He_Bag_of_Tricks_for_Image_Classification_with_Convolutional_Neural_Networks_CVPR_2019_paper.pdf.
- [11] Arjun Tuli and et al. Convnets vs. transformers: Whose visual representations are more transferable? *arXiv:2108.05305*, 2021.

240 A Appendix

241 A.1 Binary Classification

- 242 • **Fine-tuning:** Only the final fc layer was trained; all other layers frozen
- 243 • **Optimizer:** Adam
- 244 • **Learning rate:** 0.002 (fc layer)
- 245 • **Batch size:** 32
- 246 • **Epochs:** 30
- 247 • **Regularization:** None
- 248 • **Test accuracy:** 99.02%
- 249 • **Training time:** ~ 12 minutes

250 A.2 Multi-class classification

251 - Strategy 1

252 Hyper-parameter tuning (epochs and learning rate):

- 253 • 20 epochs, $\text{lr} = 1 \times 10^{-4}$: Best $l = 3$, **Val Acc** = 90.76%
- 254 • 20 epochs, $\text{lr} = 1 \times 10^{-5}$: Best $l = 2$, **Val Acc** = 89.54%
- 255 • 15 epochs, $\text{lr} = 1 \times 10^{-5}$: Best $l = 3$, **Val Acc** = **91.85%**
- 256 • 15 epochs, $\text{lr} = 1 \times 10^{-4}$: Best $l = 3$, **Val Acc** = 91.03%

257 Data augmentation:

- 258 • +RandomResizedCrop: Best $l = 1$, **Val Acc** = 92.53%
- 259 • +RandomHorizontalFlip: Best $l = 3$, **Val Acc** = **92.66%**
- 260 • +RandomRotation: Best $l = 2$, **Val Acc** = 92.53% (not used, as it decreased test performance)

261 L2 regularization:

- 262 • $\lambda = 1e-3$: Best $l = 1$, **Val Acc** = 91.71%
- 263 • $\lambda = 1e-4$: Best $l = 2$, **Val Acc** = 93.07%
- 264 • $\lambda = 1e-5$: Best $l = 1$, **Val Acc** = 91.44%
- 265 • **Conclusion:** No consistent improvement; not used in baseline.

266 Different learning rates and schedulers for different layers:

- 267 • $\text{lr}_{\text{fc}} = 1 \times 10^{-4}$, $\text{lr}_{\text{backbone}} = 1 \times 10^{-5}$: Best $l = 3$, **Val Acc** = 91.71%
- 268 • $\text{lr}_{\text{fc}} = 1 \times 10^{-3}$, $\text{lr}_{\text{backbone}} = 1 \times 10^{-5}$, $l = 4$, **Val Acc** = **94.16%**

269 BatchNorm parameters/statistics:

- 270 • Freeze params: Best $l = 4$, **Val Acc** = 92.93%
- 271 • Freeze stats: Best $l = 4$, **Val Acc** = 91.85%
- 272 • Default (not frozen): Best $l = 4$, **Val Acc** = **94.16%**
- 273 • **Conclusion:** Freezing slightly reduced validation accuracy; default used in baseline.

274 **Final test accuracy:** The best configuration (Strategy 1, $l = 4$, $\text{lr}_{\text{fc}} = 1 \times 10^{-3}$, $\text{lr}_{\text{backbone}} =$
275 1×10^{-5} , batch size 64, 15 epochs, default BatchNorm) achieved a test accuracy of **90.62%**.

276 - Strategy 2 (Gradual Unfreezing):

- 277 • Best stage: 2 or 3 (depending on run)

278 • **Best validation accuracy:** up to 93.89%

279 • Configuration: $\text{lr_fc} = 1 \times 10^{-3}$, $\text{lr_backbone} = 1 \times 10^{-5}$, batch size 64, 15 epochs,

280 ReduceLROnPlateau scheduler, default BatchNorm

281 • **Final test accuracy:** 90.02%

282 **- Imbalanced classes:**

283 • Only 20% cat breed samples: Best $l = 3$, **Test Acc** = 86.86%

284 • +Weighted loss: Best $l = 4$, **Test Acc** = 87.14%

285 • +Over-sampling: Best $l = 4$, **Test Acc** = 87.52%

286 • Weighted loss + Over-sampling: Best $l = 4$, **Test Acc** = 87.68%